

CONTEXTGRAPH: REPAIR MEMORY FOR MISSING REQUIREMENTS

Anonymous authors

Paper under double-blind review

ABSTRACT

Past repairs record requirements that a new issue may leave unstated. We study a simple repair procedure: retrieve this experience before editing and use it to complete the repair requirements. In three Django development tasks, complete repository agents produce repairs that pass official verification with or without memory, but only the history-informed repairs satisfy additional historical requirements. The added behavior preserves query parameters, removes redundant uniqueness, or honors an explicit destination timezone alongside the reported repair. Independent checks that distinguish buggy and repaired source versions expose these additional behaviors on the target repairs. On the redirect task, a generated requirement sentence alone guides a complete agent to the same repair as the sentence accompanied by an executable check and failed observations. Method-generation comparisons in Django and pytest further show that supplying history in the first call can reach behaviors that the observed ordinary sequences obtain only after revision, or leave unsatisfied. These cases show how repair memory can improve behavioral completeness beyond the requirements measured by the current task’s official tests. A further pair comparing historical requirements with edit advice produces two officially passing repairs that introduce the same database-semantic error; changing the representation alone does not ensure behavioral completeness.

1 INTRODUCTION

The issue description tells a coding agent what is currently reported as broken. It need not enumerate every behavior a complete repair should preserve. A past repair can supply a requirement absent from the new issue, allowing memory to improve a repair that already passes the target’s tests.

Consider an administrative redirect that must preserve a deployment prefix. Changing its destination from `request.path_info` to `request.path` fixes that reported fault. However, appending a slash to the resulting path still discards query parameters. A different repair of the same API records this omitted behavior and replaces path concatenation with `request.get_full_path(force_append_slash=True)`. The useful information in this experience is the requirement to retain the query, which must hold together with the deployment-prefix requirement.

This view leads to a direct repair procedure: supply the current issue and a relevant historical repair before generating code. We compare this procedure with supplying the same experience after an initial repair, and with an additional review using only the current issue. The comparison determines whether the corrective information requires a separate revision stage.

2 USING HISTORY TO COMPLETE THE REPAIR REQUIREMENTS

Supply the requirement during generation. The repair writer receives the target issue and retrieved repair memory before editing. Memory is retrieved from the source partition using frozen BM25 rankings or cosine similarity with Qwen3-Embedding-8B (Zhang et al., 2025). The historical record describes an additional behavior for the new repair to preserve. We use either the stored distilled rule or its original source issue and patch as the memory input. We also test a generated behavioral requirement sentence without the source implementation. We study two writers: a method

Table 1: Django 14404 development comparison. Every row passes official verification: 2 fail-to-pass and 323 pass-to-pass tests. Calls include the initial generation when a revision is used.

Procedure	Calls	Prefix	Query	Joint
Issue only	1	Pass	Fail	Fail
Issue + ordinary review	2	Pass	Fail	Fail
History in revision	2	Pass	Pass	Pass
History in first call	1	Pass	Pass	Pass

generator supplied with the initial implementation, and a complete SWE-agent that browses the repository, edits code, runs tools, and submits a patch. The method generator returns one repaired method while preserving its public signature and decorator; the repository agent controls its own sequence of actions.

Measure the requirements together. We immediately evaluate every generated patch with the official SWE-bench verifier (Jimenez et al., 2024). To measure the behavior contributed by the source repair, a separate writer generates a short check from its patch and relevant repaired code. A normal execution changing from false on the buggy source to true on the repaired source demonstrates the historical behavior on the supplied inputs. We then execute this check unchanged on the patched target. An issue check measures the reported requirement, and a joint input exercises it together with the historical requirement.

3 EVIDENCE: A HISTORICAL REQUIREMENT COMPLETES THE REPAIR

We use the training side of a SWE-bench Verified split with 280 source tasks, 70 development tasks, and a separate 150-task evaluation partition unused by these experiments. The development comparison pairs source Django 16612 with target Django 14404. The target issue asks for support of `FORCE_SCRIPT_NAME` in administrative redirects; the source records query preservation in the same method.

An ordinary repair omits the query. DeepSeek V4 Pro generates one initial repair from the issue and relevant initial code, including the available request-path APIs. The writer may replace `AdminSite.catch_all_view()` while preserving its signature and decorator. Its repair keeps URL resolution based on `path_info` and constructs the redirect from `request.path`. Official verification passes, and the issue check confirms that the deployment prefix is retained. The source check executes normally on this candidate and observes that both query strings are discarded.

History supplies the corrective information. Two independent revision calls start from this candidate and receive either the issue-based evidence alone or the historical issue and patch. The issue-based review retains the initial patch. The historical issue and patch lead to a replacement of path concatenation with `get_full_path(force_append_slash=True)`, preserving the prefix and query. These method-generation calls use the same model, temperature 0.7, and a 16,384-token output allowance.

The information can be supplied in the first call. We then append the identical historical issue and patch to the original initial-generation prompt, without supplying a candidate or diagnostic feedback. This one-call procedure produces a method with the same AST as the history-guided revision and passes the same checks (Table 1). The observed generation uses 4,482 provider-reported tokens, compared with 13,601 for the original initial call and historical-patch revision combined. For this development case, the additional revision stage contributes no behavioral improvement over supplying history initially.

Both requirements hold on a joint input. An independently prepared check combines the deployment prefix with repeated query parameters and percent-encoded values: `keep=a%2Fb&keep=c%20d`. The input is withheld from all writers and executed after their official verifica-

tions. The history-guided revision and the direct history generation preserve the exact prefix and query string; the initial and issue-reviewed candidates lose the query string.

A second application avoids a failed initial repair. The public issue in Pytest 7236 concerns a skipped test method whose `tearDown()` still runs under `--pdb`. Source 10081 records the corresponding class-level skip requirement. One call supplied with this history adapts the source helper to the target’s existing `__unittest_skip__` flags and checks both the method and its containing class. It passes official verification (1 fail-to-pass and 51 pass-to-pass tests), both skip checks, and a joint session that also checks a normal test’s lifecycle. The earlier ordinary initial call fails; one issue-only diagnostic revision then repairs both skip conditions using the runtime skip state. Direct history reaches these measured behaviors in the first repair call, using 8,346 provider-reported tokens compared with 25,883 for that observed two-call sequence.

The gap also occurs in a complete repository agent. We run a fresh pair of SWE-agent attempts on Django 14404, supplying either no memory or the same first-ranked source issue and patch before the agent begins. These runs use DeepSeek V4 Pro0423 through OpenRouter, temperature 0, and a 32,768-token output allowance per call; the preceding method-generation comparisons use the 0813 snapshot. Both agents browse the repository, execute tests, and submit normally. Both pass all 2 fail-to-pass and 323 pass-to-pass tests, but only the history-informed repair preserves the query and passes the joint check. The ordinary agent’s actual observations include both the existing redirect helper and the implementation that retains `QUERY_STRING`, before it edits the method. With history, the agent preserves a behavior whose implementation was already visible to the ordinary agent.

The requirement survives distillation. The existing first-ranked Flat rule explicitly describes query loss and supplies `get_full_path(force_append_slash=True)`. We run a second fixed pair of complete-agent attempts: one receives the unchanged three-entry Flat packet, while the other receives the same packet with only its first entry replaced by the original source issue and patch. The remaining entries and the delivery point are identical. Both repairs pass official verification and all three behavior checks, and their submitted patches are byte-identical. In this development comparison, either representation supplies enough information to preserve the current and historical requirements together.

The requirement need not prescribe the implementation. The distilled redirect rule includes the repair API, so we further compare two fresh agents starting from the same completed ordinary Django 14404 patch. Both receive the source-generated sentence, “The redirect location from `AdminSite.catch_all_view` includes the original query string.” One receives this sentence alone; the other also receives the unchanged generated check and its failed observations on the candidate. The agents submit byte-identical patches after 27 and 28 calls, respectively, and both pass official verification and all three behavior checks. The sentence-only agent locates `get_full_path()` in the current repository and uses it to repair the redirect. In this case, memory supplies the desired behavior while the agent finds its implementation; providing the executable feedback adds no observed behavior.

A historical requirement can extend to a neighboring operation. We run a fixed cohort of six development tasks selected for high Qwen retrieval similarity. The issue-only and Qwen Flat agents both resolve all six tasks under official verification. The following two cases identify additional historical behaviors that distinguish their repairs despite this identical official score. In a further complete-agent pair on Django 12708, the issue reports ambiguous deletion of an ordinary index when a unique constraint uses the same columns. Qwen retrieval supplies three unchanged Flat rules, with source 15732 ranked first, from a development cohort selected for high retrieval similarity. That source concerns deletion of a unique constraint sharing its column with a primary key. Both agents fix ordinary-index selection and pass official verification. The memory-informed agent also excludes primary keys when deleting unique constraints. A PostgreSQL check generated from source evidence covers redundant uniqueness beside a primary key or a field-level unique constraint. Both inputs distinguish the source versions; the unchanged check fails on the ordinary target repair and passes on the memory-informed repair, with original constraints retained. A combined input creates an ordinary index, redundant uniqueness, and a primary key on the same column. On this

PostgreSQL input, ordinary-index deletion already succeeds on the target base. Both repairs retain that behavior, but only the memory-informed repair also deletes the redundant uniqueness, retaining the primary key and original data. The useful historical information here supplies an additional operation’s requirement, beyond the operation singled out by the current failure.

History can complete a different argument of the same operation. Django 11138 asks date conversions to respect the database’s configured source timezone. Source 13363 instead requires `TruncDate` and `TruncTime` to honor an expression’s explicit destination timezone. Both complete-agent repairs pass official verification and correctly interpret a database configured as UTC+1. Only the Qwen Flat repair also honors the explicit destination. A check generated from source evidence distinguishes both source versions using a New York date aggregation and a Tokyo time aggregation; it runs unchanged on the target, failing on the ordinary repair and passing on the memory-informed repair. An independent SQLite input combines a UTC+1 database with an explicit New York destination. Only the memory-informed repair returns the correct date and time. The current issue and the retrieved history supply independent requirements for the two ends of the conversion.

Executing the historical requirement can guide a further repair. We start two fresh agents from the identical completed ordinary Django 11138 patch, with equal model, tools, and budget. One receives the issue and candidate for additional review; the other also receives the generated source check and its two failed observations, without the historical patch or the separate joint input. The ordinary review uses 47 calls and returns the original patch unchanged. The feedback-guided agent reproduces both failures, changes the two truncation methods to use `self.get_tzname()`, and submits after 43 calls. Both final patches pass official verification (1 fail-to-pass and 73 pass-to-pass tests), but only the feedback-guided revision passes both historical checks and the joint timezone input. Thus, in this intervention on a known development omission, an executed historical requirement supplies corrective information that further issue-based review does not recover.

4 APPLICABILITY IS PART OF REPAIR MEMORY

On development task Django 16819, two further complete agents receive the same three Qwen-retrieved repairs as either behavioral requirements or concrete edit advice. Both pass official verification, but both incorrectly cancel `RemoveIndex` followed by `AddIndex` whenever the name is unchanged. A SQLite diagnostic, motivated by the first agent’s intermediate actions, exposes the lost effect: removing an index on `pink` and adding the same name on `weight` should change the indexed column. Both submitted repairs reduce this sequence to no operations, leaving `pink` indexed. The original version preserves the replacement; both repairs also preserve the add-then-remove and unchanged-definition comparison cases.

The matched comparison on Xarray 7229 illustrates excessive preservation. Both agents repair the common coordinate-attribute reproduction and pass all 280 official pass-to-pass tests, but fail the official target test. The edit-advice repair retains the condition array’s coordinate attributes when both value arguments to `where` are scalars; the requirements repair already does so when only the first is scalar. Neither memory form establishes the boundary between required preservation and required change in these repairs.

That boundary can already be wrong in the historical memory. We generate source-only contracts with applicability, intended-change and preservation fields. For Xarray 4094, the contract claims a stack/unstack roundtrip preserves sample coordinates when variables have only sample dimensions. The literal official source patch restores the reported three-element dataset, but a manually chosen one-element input loses the sample dimension and coordinate. The original source raises a merge error on both inputs. The singleton satisfies the generated condition, so this package is rejected before target use. Explicit contract fields alone do not establish the scope supported by the repaired source.

5 RELATED WORK

ExpeL distills reusable insights from experience, and Getafix learns applicable edit patterns from human repairs (Zhao et al., 2024; Bader et al., 2019). ExpeRepair uses historical test demonstrations and reflective insights to generate reproduction and boundary tests (Mu et al., 2026). STAIR abstracts trajectories into hierarchical plans supplied before repair, including guidance that completes partial fixes (Xu et al., 2026b). MIRA separates SQL corrections into reusable requirements and uses database checks to guide their activation and adaptation (Liu et al., 2026). MR-Coupler derives tests from relationships between methods and validates candidate relations through amplification and mutation analysis (Xu et al., 2026a). Jiang et al. (2026) study late requirement disclosure in coding-agent sessions and controlled tasks. Our development comparison examines when a retrieved historical requirement helps preserve current and historical behaviors together, including behavior absent from the official target tests.

AI USE STATEMENT

Generative AI assisted literature review, method development, implementation, experiments, analysis, and writing.

REFERENCES

- Johannes Bader, Andrew Scott, Michael Pradel, and Satish Chandra. Getafix: Learning to fix bugs automatically, 2019. URL <https://arxiv.org/abs/1902.06111>.
- Bowen Jiang, Haowei Cheng, Yuhong Fu, Anne Koziolok, Jialong Li, and Weixing Zhang. Requirements after the first edit: Mining late requirement emergence and rework in real-world coding-agent sessions, 2026. URL <https://arxiv.org/abs/2609.03028v1>. Version 1.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024. URL <https://arxiv.org/abs/2310.06770>.
- Yining Liu, Chenyu Yang, Boyan Li, Rui Mao, and Yuyu Luo. MIRA: Evidence-verified repair memory for text-to-sql correction, 2026. URL <https://arxiv.org/abs/2608.06950v1>.
- Fangwen Mu, Junjie Wang, Lin Shi, Song Wang, Shoubin Li, and Qing Wang. ExpeRepair: Dual-memory enhanced LLM-based repository-level program repair, 2026. URL <https://arxiv.org/abs/2506.10484v2>. Version 2; accepted at FSE 2026.
- Congying Xu, Hengcheng Zhu, Songqiang Chen, Jiarong Wu, Valerio Terragni, and Shing-Chi Cheung. MR-Coupler: Automated metamorphic test generation via functional coupling analysis, 2026a. URL <https://arxiv.org/abs/2604.10126>.
- Yisen Xu, Jiayuan Zhou, Ruiqi Pan, and Tse-Hsun Chen. Reusing past repairs through hierarchical trajectory abstraction for coding agents, 2026b. URL <https://arxiv.org/abs/2607.29658v1>. Version 1.
- Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. Qwen3 embedding: Advancing text embedding and reranking through foundation models, 2025. URL <https://arxiv.org/abs/2506.05176v3>.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners, 2024. URL <https://arxiv.org/abs/2308.10144v3>.